01

BASES
DE DONNÉES
MODERNES

Apprendre
Comprendre
Construire
demain

CHAPITRE 2

Bases de données orientées documents : MongoDB

Structure des données • Requêtes • Modélisation • CRUD

CHAPITRE 2

Partie 1
Structure des données dans MongoDB

Comprendre comment MongoDB organise et représente les données.

Le modèle orienté documents

MongoDB stocke les données sous forme de documents

Relationnel

Tables, lignes et colonnes.

MongoDB

Documents sous formatJSON.

Idée clé

Un document décrit un objet complet.

i | Un document MongoDB peut contenir des champs simples, des tableaux et des objets imbriqués.

Organisation générale de MongoDB

?

L'université abandonne ses fiches papier et ses fichiers Excel. Elle vous confie la création de sa nouvelle plateforme. Votre interlocutrice : Mme Bennani, responsable de la scolarité. Elle va vous envoyer des demandes tout au long de la séance. À vous de trouver comment les stocker.

Mme Bennani : *« Voici la fiche d'inscription de Sara Alaoui : âge 21 ans, filière Informatique. Enregistrez-la. »*

DB Base de données

universite

C Collection

etudiants

{} Document

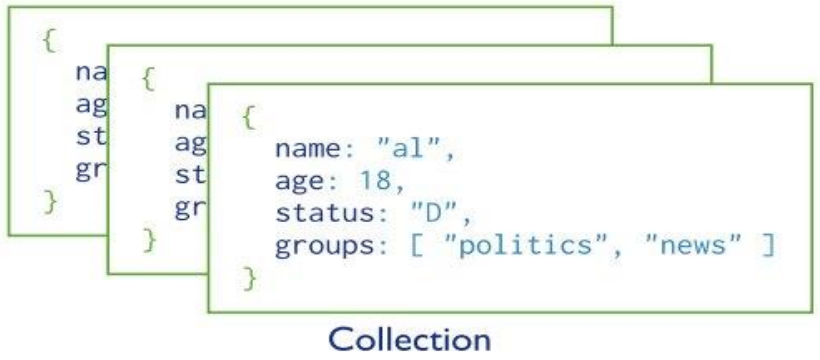
{ nom: "Sara" }

x Champ : valeur

nom : "Sara"

Exemple d'organisation

universite
  etudiants
    { nom : "Sara", age : 21, filière : "informatique" }
    { nom : "Yassine", age : 22 , filière : "informatique" }
  professeurs
  cours



Le champ _id et ObjectId

Chaque document possède un identifiant unique

Sans préciser _id

```
db.etudiants.insertOne({
    nom: "Sara",
    age: 21 })

// MongoDB ajoute automatiquement :
{ _id: ObjectId("..."), nom: "Sara",
age: 21 }
```

Types possibles pour _id

```
{ _id: ObjectId("..."), nom: "Sara"
}
{ _id: 1, nom: "Yassine" }
{ _id: "ETU001", nom: "Amine" }
```

i | _id est obligatoire et unique dans une collection. ObjectId est le type généré automatiquement si aucun _id n'est fourni.

Les principaux types de données

MongoDB permet de représenter des données riches

? **Mme Bennani :** *« Les entreprises veulent connaître les compétences des étudiants. Sara maîtrise Java, Python et MongoDB. »*

Mme Bennani : *« Il me faut aussi l'adresse pour envoyer les diplômes : rue, ville, code postal. »*

Type	Exemple
String	"Sara"
Number	21 ou 15.7
Boolean	true / false
Date	new Date()
Array	["Java", "Python"]
Object	{ ville: "Casa", CP: "2000" }
ObjectId	ObjectId("...")

Document combinant plusieurs types

```
{
  nom: "Sara",
  age: 21,
  actif: true,
  competences: ["Java","Python","MongoDB"],
 adresse: {rue: "...",
           ville: "Casablanca",
           codePostal: "20000" }

}
```

Documents imbriqués et tableaux

Représenter naturellement des données complexes

Exemple

```
{
  nom: "Sara Alaoui",
  adresse: {
    rue: "12 rue Ibn Sina",
    ville: "Casablanca",
    codePostal: "20000"
  },
  competences: ["Java", "Python", "MongoDB"]
}
```

{} **Document imbriqué**

Un objet stocké à l'intérieur d'un document.

[] **Tableau**

Une liste de valeurs ou de sous-documents.

i Ces structures rendent le modèle documentaire très pratique pour les données semi-structurées.

JSON et BSON

Deux termes à distinguer dès le début

{} **JSON**

Format lisible, proche de ce que l'on écrit dans les exemples et dans les applications web.

B **BSON**

Format binaire utilisé par MongoDB pour stocker les documents efficacement.

¡ | À retenir : on écrit souvent les exemples comme du JSON, mais MongoDB stocke réellement les documents en BSON.

Schéma flexible

? **Des profils différents**

Mme Bennani : « *Yassine est en alternance, il faut son entreprise et son rythme. Amine est en stage, il faut son tuteur et ses dates. Les autres n'ont ni l'un ni l'autre.* »

Document 1 · Alternance

```
{
  nom: "Yassine El Amrani",
  age: 22,
  filiere: "Informatique",
  alternance: {
    entreprise: "OCP Digital",
    rythme:
      "3j entreprise / 2j école"
  }
}
```

Document 2 · Stage

```
{  nom: "Amine Tazi",
  age: 23,
  filiere: "Informatique",
  stage: {
    tuteur: "M. Berrada",
    dateDebut:
      new Date("2026-02-01"),
    dateFin:
      new Date("2026-05-31")
  }
}
```

Document 3 · Classique

```
{
  nom: "Imane Chraibi",
  age: 20,
  filiere: "Informatique"
}
```

Champs communs : nom, age, filiere · **en orange : les champs propres à chaque profil**

¡ Schéma flexible ne signifie pas absence de conception : il faut garder une structure cohérente et documenter le modèle choisi.

Correspondances avec le modèle relationnel

Un repère rapide pour passer de SQL à MongoDB

Relationnel	MongoDB
Base de données	Base de données
Table	Collection
Ligne	Document
Colonne	Champ
Clé primaire	_id
JOIN	Imbrication / référence

Caractéristiques principales de MongoDB

Une base documentaire flexible et adaptée aux applications modernes

- **Sans schéma fixe**
  chaque document peut avoir des champs différents
- **Types de données riches**
  chaînes, booléens, tableaux, objets imbriqués
- **Indexation et agrégation puissantes**
  recherches et traitements efficaces
- **Scalabilité horizontale**
  répartition des données avec le sharding
- **Format BSON**
  optimisé pour le stockage et la vitesse

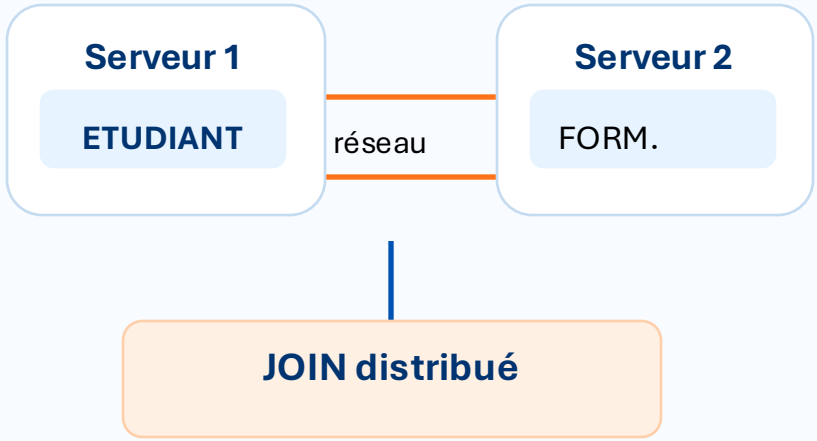
Avantage	Description
Flexibilité du schéma	Les documents peuvent évoluer sans migration
Lecture rapide	Accès direct au document complet
Regroupement logique	Données liées dans un même objet
Intégration simple	Compatible avec les langages web

💡 Une solution moderne, flexible et performante pour les applications d'aujourd'hui.

Scalabilité horizontale : BD relationnelle vs MongoDB

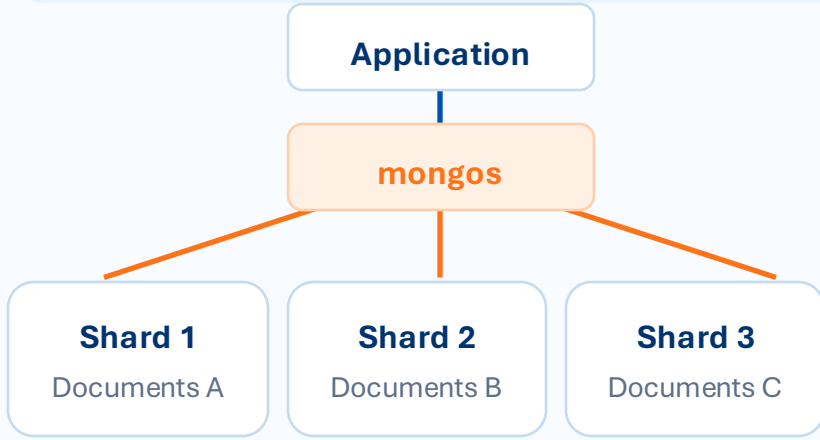
Même objectif : répartir la charge sur plusieurs serveurs — mais le fonctionnement n'est pas le même.

BD relationnelle distribuée



Les données liées peuvent se trouver sur plusieurs serveurs.

MongoDB avec sharding



Shard key : oriente les documents vers les shards.

mongos route les requêtes sans exposer les shards à l'application.

Idée clé : le relationnel peut aussi scaler horizontalement, mais les JOIN répartis peuvent coûter cher ; MongoDB répartit des documents plus autonomes via le sharding.

Préparer le contexte de travail

Choisir une base, créer une collection, afficher le contexte et changer de base

Commandes de départ

```
// 1. Choisir une base (ou la créer à la première insertion)
use universite

// 2. Vérifier la base actuelle
db
// ou : db.getName()

// 3. Créer une collection dans cette base
db.createCollection("etudiants")

// 4. Afficher les collections de la base actuelle
show collections

// 5. Passer d'une base de données à une autre
use eCommerce
```

CHAPITRE 2

Partie 2
Opérations CRUD dans MongoDB

Créer, consulter, modifier et supprimer des documents.

CRUD : principe général

Les quatres opérations fondamentales

C

Create

Ajouter de nouveaux documents.

R

Read

Consulter un ou plusieurs documents.

U

Update

Modifier un ou plusieurs documents.

D

Delete

Supprimer un ou plusieurs documents.

Méthodes principales

Create → insertOne(), insertMany()
Read → find(), findOne()
Update → updateOne(), updateMany(), replaceOne()
Delete → deleteOne(), deleteMany()

CREATE — Ajouter des documents

Insérer un ou plusieurs documents

Exemples

```
// Un document
db.etudiants.insertOne({
  nom: "Sara", age: 21,
  filiere: "Informatique"
})

// Plusieurs documents
db.etudiants.insertMany([
  { nom: "Sara", age: 21 },
  { nom: "Yassine", age: 22 }
])
```

- insertOne()

  ajoute un seul document

- insertMany()

  ajoute plusieurs documents

- _id généré

  ObjectId créé si _id absent

READ — Consulter les documents

Lire un ou plusieurs documents

Exemples

```
// Tous les documents
db.etudiants.find()

// Un seul document
db.etudiants.findOne({ nom: "Sara" })

// Avec filtre
db.etudiants.find({ age: { $gte: 20 } })
```

- find()

  peut retourner plusieurs documents

- findOne()

  retourne un seul document

- Filtres

  les conditions on va les traiter en détail dans la partie 3

UPDATE — Modifier un document

Exemple

```
db.etudiants.updateOne(
  { nom: "Sara" },
  {
    $set: {
      age: 22
    }
  }
)
```

- **1er argument**

  le **filtre** : quel document faut-il trouver ?

- **2e argument**

  la modification : que faut-il changer ?

- **updateOne()**

  modifie le premier document correspondant au filtre

Les opérateurs de mise à jour

Les principaux opérateurs

Opérateur	Rôle
$set	ajouter un champ ou modifier sa valeur
$unset	supprimer un champ
$inc	augmenter ou diminuer une valeur numérique
$mul	multiplier une valeur numérique
$min	modifier seulement si la nouvelle valeur est plus petite
$max	modifier seulement si la nouvelle valeur est plus grande
$rename	renommer un champ
$currentDate	enregistrer la date/heure actuelle
$setOnInsert	ajouter des valeurs seulement si upsert crée un document

Opérateurs pour les tableaux

Opérateur	Rôle
$push	ajouter un élément
$addToSet	ajouter seulement s'il n'existe pas déjà
$pull	supprimer les éléments correspondant à une condition
$pop	supprimer le premier : **{ $pop: { competences: 1 } }** ou le dernier élément : **{ $pop: { competences: -1 } }**
$pullAll	supprimer plusieurs valeurs

Les opérateurs de mise à jour

$elemMatch : Plusieurs conditions sur le même élément d'un tableau

? | **Mme Bennani :** « *Quels étudiants ont au moins une note comprise entre 10 et 15 ?* »

Le document

```
{ nom: "Sara",
  notes: [8, 18]
}

8 < 10 et 18 > 15 :
aucune note entre 10 et 15
```

Sans $elemMatch

```
db.etudiants.find({
  notes: {
    $gte: 10,
    $lte: 15 } })
```

Sara trouvée à tort

18 vérifie ≥ 10, 8 vérifie ≤ 15 : chaque condition est remplie par une note différente.

Avec $elemMatch

```
db.etudiants.find({
  notes: { $elemMatch: {
    $gte: 10,
    $lte: 15 } } })
```

Sara non trouvée

Il faut une seule note qui soit à la fois ≥ 10 ET ≤ 15.

À retenir Une seule condition sur un tableau → `$elemMatch` inutile.

Plusieurs conditions qui doivent porter sur le **même élément** → `$elemMatch` obligatoire.

Modifier plusieurs documents et remplacer

updateMany() et replaceOne()

Exemples

```
// Modifier plusieurs documents
db.etudiants.updateMany(
  { filiere: "Informatique" },
  { $set: { actif: true } }
)

// Remplacer un document
db.etudiants.replaceOne(
  { _id: 1 },
  { nom: "Sara",
    age: 22,
    filiere: "Data"
  }
)
```

- **updateMany()**
  modifie tous les documents correspondant au filtre
- **replaceOne()**
  remplace complètement le document, contrairement à $set avec updateOne ou updateMany
  ReplaceOne modifie la structure de l'ancien document
- **Attention**
  un filtre vide {} peut modifier toute la collection

DELETE — Supprimer des documents

Exemples

```
// Un document
db.etudiants.deleteOne({ nom: "Sara" })

// Plusieurs documents
db.etudiants.deleteMany({ actif: false })

// Cas dangereux
db.etudiants.deleteMany({})
```

- **deleteOne()**

  supprime le premier document correspondant

- **deleteMany()**

  supprime tous les documents correspondant au filtre

- **Attention**

  {} supprime tous les documents de la collection

Upsert et résultats des opérations

- **upsert: true**

  si le document existe : UPDATE ; sinon : INSERT

Upsert

```
db.etudiants.updateOne(
        { email: "sara@example.com" },
        { $set: { nom: "Sara", age: 21 } },
        { upsert: true }
)
```

```
test> db.etudiants.updateOne({email:"sara@exemple.com"},{$set:{nom : "Sara2"}},{upsert:true})
{
  acknowledged: true,
  insertedId: ObjectId('6abb5d339db76cea48b11c0e'),
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 1
}
```

- **Suppression**

  résultat : deletedCount

- **Insertion**
  - **acknowledged** : indique si MongoDB a confirmé la prise en compte de l'opération d'écriture.
    - **true** : MongoDB confirme l'écriture.
    - **false** : aucune confirmation n'a été demandée (avec `writeConcern: { w: 0 }`)
  - **insertedId** : identifiant `_id` du document inséré avec `insertOne()`.
  - **insertedIds** : identifiants `_id` des documents insérés avec `insertMany()`.

```
test> db.etudiants.insertOne({nom:"sara", email: "sara@example.com"})
{
  acknowledged: true,
  insertedId: ObjectId('6abb5bad3fc2e978faf7dc1d')
}
```

- **Mise à jour**
  - **acknowledged** : indique si MongoDB a confirmé l'opération.
  - **matchedCount** : nombre de documents correspondant au filtre.
  - **modifiedCount** : nombre de documents réellement modifiés.
  - **insertedId** :
    - contient l'`_id` du nouveau document si `upsert` a provoqué une insertion ;
    - vaut `null` si un document existant a été trouvé.

Travail préparatoire

Constituer la base de la plateforme

? | **Mme Bennani :** *« Enregistrez ces 11 fiches avant la prochaine séance, nous en aurons besoin ! »*

Nom	Âge	Filière	Moy.	Ville	Code postal	Compétences	Téléphone
Sara Alaoui	21	Informatique	15.7	Casablanca	20000	Java, Python, MongoDB	0612345678
Yassine El Amrani	22	Informatique	13.2	Rabat	10000	Java, SQL	0623456789
Amine Tazi	23	Informatique	16.8	Casablanca	20250	Python, MongoDB, Docker	—
Imane Chraibi	20	Informatique	14.1	Témara	12000	HTML, CSS, JavaScript	0634567890
Salma Bennis	19	Data Science	17.4	Casablanca	20100	Python, MongoDB, Machine Learning	0645678901
Omar Fassi	24	Data Science	12.5	Fès	30000	Python, SQL	—
Khadija Idrissi	20	Réseaux	16.2	Rabat	10020	Linux, Cisco	0656789012
Mehdi Lahlou	21	Génie Logiciel	11.9	Marrakech	40000	Java, Spring, MongoDB	0667890123
Nora Ziani	22	Réseaux	14.8	Casablanca	20070	Linux, Python	—
Samir Ouazzani	25	Informatique	15.3	Casablanca	20340	JavaScript, Node.js, MongoDB	0678901234
Hamza Benjelloun	22	Informatique	12.4	Casablanca	20200	C, Java	0689012345

1. `adresse` : { `ville`, `codePostal` } imbriqué
2. `competences` → tableau · `age`, `moyenne` → nombres
3. `telephone` « — » → **pas de champ**
4. Sara : `insertOne()` · les 10 autres : `insertMany()`

Yassine : alternance (OCP Digital, 3j/2j)
Amine : stage (M. Berrada, 01/02 → 31/05/2026)
→ documents imbriqués, dates en `Date`
✓ `countDocuments()` → 11

CHAPITRE 2

Partie 3
Langage de requête dans MongoDB

Savoir exprimer une recherche, filtrer et sélectionner les données.

Introduction à MQL

MongoDB Query Language

Première requête

```
db.etudiants.find({
  filiere: "Informatique"
})
```

db

Base courante

etudiants

Collection

find()

Méthode

{ filiere: "Informatique" }

Filtre de recherche

i | Dans la Partie 3, find() sert surtout à apprendre à construire des filtres et des conditions.

Filtres simples

Rechercher tous les documents ou selon une égalité

Exemples

```
// Tous les documents
db.etudiants.find()

// Une condition
db.etudiants.find({ age: 21 })

// Plusieurs critères
db.etudiants.find({
  age: 21,
  filiere: "Informatique"
})
```

- **Filtre vide**
  retourne tous les documents
- **Égalité**
  champ : valeur
- **Plusieurs critères**
  équivalent à un ET logique implicite

Opérateurs de comparaison

Exprimer des conditions plus riches

Opérateur	Signification
$eq	égal
$ne	différent
$gt	supérieur à
$gte	supérieur ou égal
$lt	inférieur à
$lte	inférieur ou égal
$in	appartient à une liste
$nin	n'appartient pas à une liste

Exemple

```
db.etudiants.find({
  age: { $gte: 20, $lte: 25 }
})

// Équivalent conceptuel :
// WHERE age BETWEEN 20 AND 25

db.collection.find({ champ: { $in:
[valeur1, valeur2, ...] } })
```

i | Les opérateurs de comparaison sont placés dans un document associé au champ concerné.

Opérateurs logiques

Combiner plusieurs conditions

$and

∧

Toutes les conditions doivent être vraies.

$or

∨

Au moins une condition doit être vraie.

$not

¬

Inverse une condition.

$nor

∅

Aucune condition ne doit être vraie.

Exemple avec $or

```
db.etudiants.find({
  $or: [
    { filiere: "Informatique" },
    { filiere: "Réseaux" }
  ]
})
```

Documents imbriqués et tableaux

Interroger les structures complexes

Notation pointée (Dot notation) et tableaux

```
{
 nom: "Sara",
 adresse: { ville: "Casablanca" },
 competences: ["Java", "Python", "MongoDB"]
}

// Document imbriqué
db.etudiants.find({ "adresse.ville": "Casablanca" })

// Tableau
db.etudiants.find({ competences: "MongoDB" })
```

- **Dot notation**
  accéder à un champ imbriqué : caracteristiques.ram
- **Tableau**
  rechercher une valeur présente dans la liste
- **Intérêt**
  exploiter directement la richesse du document

Projection

La **projection** permet de choisir quels **champs** d'un document seront **retournés par une requête**.

Exemple

```
db.etudiants.find(
  {},
  { nom: 1, filiere: 1, _id: 0 }
)
```

✓ **1**
Afficher le champ

× **0**
Masquer le champ

- **Premier document**
  le filtre de recherche
- **Deuxième document**
  la projection des champs
- **_id**
  affiché par défaut sauf si _id: 0

Trier, limiter et paginer

Organiser les résultats retournés

Méthodes principales

```
// Trier par âge croissant
db.etudiants.find().sort({ age: 1 })

// Trier par moyenne décroissante
db.etudiants.find().sort({ moyenne: -1 })

// Limiter à 5 résultats
db.etudiants.find().limit(5)

// Ignorer 5 résultats
db.etudiants.find().skip(5)
```

sort()

1 = ordre croissant, -1 = ordre décroissant

limit() et skip()

Utile pour afficher une page de résultats.

i Exemple concret : afficher les 5 meilleures moyennes avec sort({ moyenne: -1 }).limit(5).

Requêtes complémentaires

Exister, chercher par motif, compter et distinguer

Exemples

```
// Champ existant
db.etudiants.find({ telephone: {
$exists: true } })

// Nom qui commence par S
db.etudiants.find({ nom: { $regex: "^S"
} })

// Compter
db.etudiants.countDocuments({ filiere:
"Informatique" })

// Valeurs distinctes
db.etudiants.distinct("filiere")
```

- **$exists**
  utile avec un schéma flexible
- **$regex**
  recherche selon un motif
- **countDocuments()**
  compter les documents correspondants
- **distinct()**
  obtenir les valeurs différentes

Les expressions régulières (regex)

Chercher ce qui ressemble à un motif, et non un mot exact

Les symboles de base

Symbole	Signification	Exemple	Trouve
^	début du texte	/^Al/	Ali, Alaoui
$	fin du texte	/ma$/	Salma, Asma
.	tout caractère	/S.ra/	Sara, Sora
*	0 fois ou plus	/ab*c/	ac, abbc
+	1 fois ou plus	/ab+c/	abc, abbc
?	0 ou 1 fois	/colou?r/	color, colour
[ ]	un caractère parmi	/[MN]ora/	Mora, Nora
[^]	un caractère sauf	/[^0-9]/	a, B, %
|	ou	/Rabat|Fès/	Rabat, Fès
{n}	exactement n fois	/^\d{5}$/	10000

Les options

Option	Effet
i	ignore la casse
m	multiligne
x	ignore les espaces

Comment l'utiliser dans MongoDB

1- Formelittérale, entre / /

```
db.etudiants.find({ nom: /^S/i })
```

2- Opérateur $regex (motif en chaîne)

```
db.etudiants.find({ nom: {
    $regex: "^S", $options: "i" } })
```

Dans un $in, seule la forme littérale :

```
{ nom: { $in: [/^A/, /^S/] } }
```

! **Piège du point :** /gmail.com/ trouve aussi gmailXcom
Vrai point : /gmail\.com/ (échapper avec \)

! /ali/ contient · /^ali/ débute
/^ali$/ est exactement « ali »

Travail à faire

Première manipulation des requêtes

? | **Mme Bennani :** *« À vous de répondre à mes questions ! Écrivez la requête qui affiche : »*

Filtrer et comparer

1. les étudiants de la filière Data Science ;
2. les étudiants d'Informatique âgés de 22 ans ;
3. les étudiants ayant une moyenne inférieure à 13 ;
4. les étudiants âgés de 20 à 22 ans (inclus) ;
5. les étudiants de Réseaux ou de Data Science, avec `$in` ;
6. les étudiants de Réseaux **ou** ayant une moyenne supérieure à 16 ;
7. les étudiants qui habitent à Rabat ;

Tableaux, affichage et compléments

8. les étudiants qui maîtrisent Java ;
9. seulement le nom et la filière de tous les étudiants, sans `_id` ;
10. les 3 étudiants ayant les moyennes les plus faibles ;
11. les étudiants qui n'ont pas de téléphone ;
12. les étudiants dont le nom commence par M ;
13. le nombre d'étudiants qui habitent à Casablanca ;
14. les différentes villes des étudiants.